

Kasparro

AI Resume Screening

Submitted by Aryabrat Mishra | Roll number 22053670
aryabrat.mishra1@gmail.com | 7 October 2026

A reproducible screening pipeline with deterministic eligibility, evidence-backed scores, optional semantic annotation and graceful public GitHub enrichment.

Full-batch outcome	Observed result
Input resumes	50
Successfully parsed	50
Eligible / rejected	40 / 10
Unreadable / duplicates	0 / 0
PDF pages / unique embedded links	62 / 259
Tests	27 passed
Evidence spans validated	480

What these results establish

The application processed the entire supplied folder and its final JSON passed schema, arithmetic, ordering, category-cap and evidence-provenance checks. These checks establish operational consistency. They do not establish ranking accuracy or verify candidates' resume claims.

No trusted eligibility labels or gold ranking were supplied. True screening or ranking accuracy cannot be calculated from this dataset. The manual review described on page 3 is a spot-check, not measured accuracy.

Privacy: this report contains aggregate data only. Candidate names, contact details, URLs, project quotations and per-candidate rankings remain in private local artifacts.

Method and scoring

Ingest -> extract text and links -> deterministic hard filter -> evidence scoring -> public GitHub bonus -> stable ranked JSON.

A candidate needs a genuine Python mention plus an AI implementation claim. Skills or coursework alone do not prove an AI project. Java, JavaScript and React are compatible supporting stacks. Custom ML/NLP/computer-vision implementations count as equivalent AI exposure; agentic and RAG depth receive additional points.

Category	Cap	Implemented sub-rules
AI / agentic / RAG	40	AI implementation 8; retrieval 6; embeddings 4; tools 5; state 5; orchestration 5; evaluation 4; processing 3.
Python / backend	30	Python implementation 10 (skill only 2); Python backend framework 8; async 4; PostgreSQL 4; Redis 4.
Cloud / deployment / full stack	15	Cloud 5; containers 4; deployment/CI 4; supporting frontend 2.
GitHub	10	Recent engineering events 0-5 plus maintained/relevant public repositories 0-5.
Engineering depth	5	One each for testing, architecture, performance systems, observability, failure handling.

The brief supplies category weights and qualitative guidance; the sub-rules above are explicit engineering choices. A feature earns points once per resume and requires implementation evidence. The total is $\max(0, \text{sum of category scores minus penalties})$.

Project-quality penalties

Thin LLM/API claims lacking workflow, data processing, retrieval, state, backend, evaluation or product logic lose 10 points when no deeper AI work is evident, or 5 when other AI work is stronger. Explicit tutorial ownership limitations lose 5. Penalties cap at 15. Project paragraphs are considered together; ambiguous quality judgments remain a human-review concern.

Evidence records carry exact normalized-text quotes and offsets. GitHub contributions instead trace to retrieved event/repository snapshots. Resume performance percentages are recorded as claims and are not treated as independently validated outcomes.

Validation and review

The ZIP contains 50 PDFs and no labels, README or sample submission. All extracted resumes were inspected for Python/AI signals, page readability and link availability. Byte and normalized-text hashes found no duplicates.

Extraction defects addressed

A blank trailing page initially marked one otherwise readable document as failed. Visual inspection confirmed it was blank; the parser now retains the resume and records a low-text-page warning. Spaced/fragmented glyphs and multi-column documents use a second extraction path, with block-order preservation where appropriate. One overlaid-text document required preferring its readable text layer. The final batch has 50 parsed documents across 62 pages, including that blank page.

Automated verification

Twenty-seven tests pass. Independent final-file validation checked 480 exact evidence/penalty spans, all 50 output records, score arithmetic, category caps, descending order, rank/eligibility consistency, duplicate exclusion, positive GitHub provenance and public-result privacy controls. Synthetic tests cover malformed files, embedded links, mixed stacks, skill-only claims, present-tense projects, penalties, metadata boundaries, cache/network failures and optional model failures.

Manual spot-check

A purposive subset of 15 resumes was reviewed against extracted text and the brief. Case IDs and evidence notes are recorded in the private `manual_review.json`. The subset covers rejected-only stacks, mixed stacks, ML equivalents, shallow/deep projects, unusual prose, multi-column extraction, blank pages and ambiguous profile ownership. Final eligibility disagreements: 0. This review is by the implementing agent, is not independent or random, and cannot support a population accuracy estimate.

Ranking limitations

Keyword variants and paragraph grouping can still miss or misattribute engineering detail. Scores are a documented heuristic, not a calibrated measure of candidate ability. The spot-check did not establish a gold numeric score or total ordering. Independent labels, project ownership verification and reviewer ranking judgments are required before measuring screening or ranking quality.

Optional LLM status: no real model call was made because no configured credentials were available. Structured-output validation and failure handling were tested; live provider compatibility remains unverified.

GitHub, privacy and handoff

GitHub enrichment status	Profiles
Verified retrieval	30
Unavailable / rate limited	11
Missing profile	7
Ambiguous owner links	2

Public requests inspect at most 100 recent events and 100 repositories. Engineering events within 90 days earn up to 5 points. Non-fork, non-archived repositories pushed within 180 days earn one point each, plus one for Python/AI relevance, capped at 5. Activity windows use 7 October 2026. Snapshots record retrieval provenance and remain private.

Missing, private, ambiguous or unavailable data never fails eligibility. Unknown GitHub merit is null; only its awarded bonus is zero. Unverified candidates have a score interval reflecting a possible additional 10 points. With incomplete enrichment, close rankings should be reconsidered when verified evidence becomes available.

Run and review

```
python -m pip install -r requirements.txt
python main.py --input resumes --output private/batch/results.json
python tools/export_public_results.py private/batch/results.json output/results.json
python -m unittest discover -s tests -v
python tools/validate_public.py output/results.json
```

Use `--offline` to disable GitHub and `--cache` with `--as-of` to reuse a private snapshot. The synthetic examples require no private dataset. Optional `--llm` explicitly sends resume text to the configured provider; it cannot override eligibility, scores or ranks.

Delivered artifacts

The public repository contains modular source, tests, configuration examples, README, synthetic examples, aggregate outputs, privacy-safe results for all 50 candidates and this report. Public results retain ranks, eligibility, score breakdowns and generic evidence rules while omitting names, contacts, filenames, profile identifiers, URLs and exact quotations. The private submission ZIP contains full evidence and manual-review notes without raw PDFs.

Submission repository

The public submission targets the user-provided GitHub repository: [codebyArya-bit / AI-Resume-Screening-Kasparro-Assignment-](#). README documents setup, scoring decisions and privacy exclusions. Full candidate results and supporting evidence are supplied separately in the private submission ZIP for authorized reviewers.